Assignment 1.2: AI Agents and AI Literature

Francisco Gutierrez

Question 1 documents the literature collection workflow. Question 2(a) contains the agent implementation; Questions 2(b) and 2(c) define its proposed goal and future functionality. Agent execution is not required. An optional execution attempt returned a Gemini 503 availability error; no successful agent actions are claimed.

Submission guide: This PDF answers Questions 1(a), 1(b), 2(b), and 2(c). Question 2(a), including code and its recorded execution outcome, is provided in the separate HTML notebook export.

Question 1: AI Literature Review

Question 1 | Search, Collect, Process

Purpose: Identify at least five technology information sources, explain why they were selected, and document a repeatable workflow for collecting and reviewing recent articles.

Collection tools: RSS Guard is installed and configured with five feeds. ChatGPT and the optional CSV tracker support the manual review stage.

Question 1(a): Five information sources and rationale

I plan to follow five sources covering different parts of the technology industry, particularly artificial intelligence, machine learning, software development, and cybersecurity.

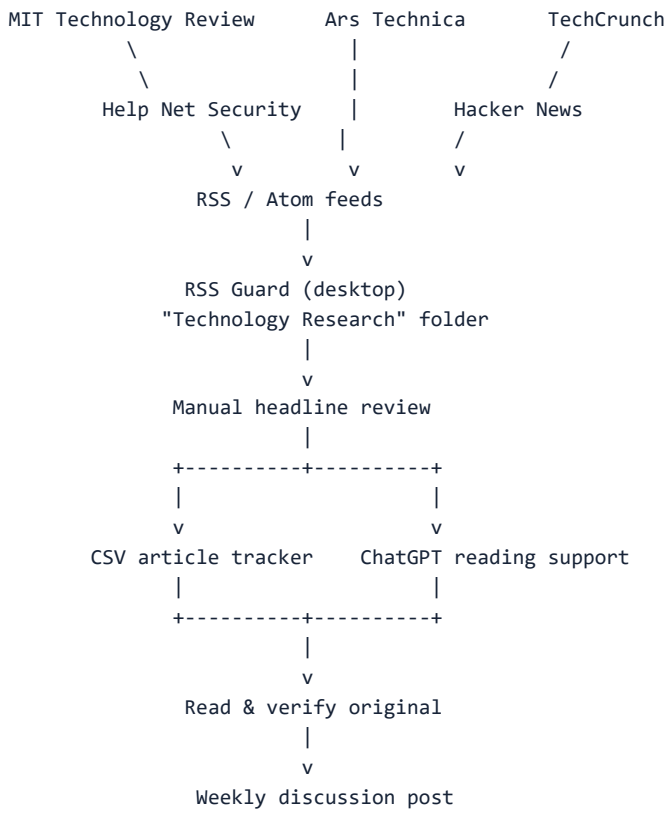
Information source	Website	Why I chose it
MIT Technology Review	https://www.technologyreview.com/	Covers AI research and emerging technologies, which directly relate to my studies in applied AI and machine learning.
Ars Technica	https://arstechnica.com/	Provides in-depth reporting on computing, software development, and technical issues relevant to my programming background.
TechCrunch	https://techcrunch.com/	Helps me follow startups, new products, and the business applications of technology.
Help Net Security	https://www.helpnetsecurity.com/	Covers cybersecurity threats, vulnerabilities, and AI-related security concerns.
Hacker News	https://news.ycombinator.com/	Aggregates articles and community discussions, exposing me to topics developers and researchers find useful.

Together, these provide research-oriented reporting, industry news, cybersecurity coverage, and practitioner perspectives. Hacker News is an aggregator, so I will verify claims using the original linked article.

Question 1(b): Collection workflow and technical architecture

I configured **RSS Guard**, a free desktop RSS reader, with five RSS feeds in a **Technology Research** folder and fetched their article headlines. This collects new technology articles locally without an online news-reader account and lets me review sources in one application. I will manually select promising articles and record them in a tracker. ChatGPT can help me understand unfamiliar terminology and summarize what I read, but I will verify its output against each original article.

Architecture diagram



Automation boundary: RSS Guard downloads feed updates when the application is running and feeds are refreshed. Reviewing articles, choosing a topic, confirming its publication date, and writing the final discussion remain manual. Not every site offers one obvious feed URL; confirm the working RSS/Atom feed for each publication and use its official website as a fallback when necessary.

Collection configuration and review frequency

The five RSS endpoints below were checked and returned parseable RSS feeds with article entries. They are also packaged in `Technology_Research_Feeds.opml`, which groups the subscriptions under **Technology Research** for import into RSS Guard.

Source	RSS address
MIT Technology Review	https://www.technologyreview.com/feed/
Ars Technica	https://feeds.arstechnica.com/arstechnica/index
TechCrunch	https://techcrunch.com/feed/
Help Net Security	https://www.helpnetsecurity.com/feed/
Hacker News	https://news.ycombinator.com/rss

Completed configuration: I imported `Technology_Research_Feeds.opml` through **Accounts > the local RSS account > Import feeds**, then selected **Fetch all**. The Technology Research folder contains all five subscriptions. Verification on October 26, 2026 found **90 saved articles** in the local RSS Guard database, and I confirmed that article headlines appeared in the application. This verifies collection, not that every saved article meets the seven-day publication requirement; dates will be checked when selecting the weekly reading.

Review schedule: I will refresh and review headlines every **Monday, Wednesday, and Friday**, for **10-15 minutes** each session. I will check feed failures during each review and verify my source mix monthly.

Selection and processing: I will record promising articles in a CSV tracker with source, title, original publication date, URL, and notes. Before each weekly discussion, I will choose an article published within the preceding seven days, read the original, verify the date, and write my own response with citations. ChatGPT will serve as a reading aid; I will check its explanations against the original article.

Technical boundary: RSS Guard refreshes feeds while running. Article selection, date verification, tracker entry, and discussion writing remain manual. The tracker is stored locally; ChatGPT only receives text that I choose to share.

Question 1(b): Article tracking

The editable notebook includes an optional Python CSV tracker. It records source, title, publication date, URL, and notes, and filters candidate articles by publication date. The tracker can also be maintained directly in a spreadsheet. I will verify original publication dates because an updated date does not necessarily indicate a new article.

Publication-date check

The optional notebook tracker compares recorded publication dates with the review date. Original publication dates must still be checked on the source websites.

Question 1(b): Workflow summary

My collection workflow uses five technology news sources configured in **RSS Guard**, with an optional Python-based article tracker and ChatGPT as a reading assistant. RSS Guard has fetched the feeds locally, so I do not need an account with an online RSS service. I plan to review its headlines on **Monday, Wednesday, and Friday** for about **10–15 minutes** each time. I will record promising articles in a CSV file with their titles, sources, publication dates, links, and notes. I can then use the Python tracker to identify articles from the preceding seven days and choose one for each week's discussion. I will verify facts and dates from the original websites. This workflow helps me stay organized while following AI, machine learning, software development, and cybersecurity news.

Links to workflow tools

- RSS Guard: https://rssguard.com/
- ChatGPT: https://chatgpt.com/
- Python documentation (CSV module): https://docs.python.org/3/library/csv.html

Question 2(b): Technical goal and weighted definitions of success

Goal. My study-planning agent should turn assignment deadlines, course requirements, and available calendar windows into a feasible, traceable study plan. For a planning horizon of the next seven days, it should prioritize the earliest deadlines, create useful tasks, reserve available study time, and save a plan that accurately reflects the actions its tools completed. This is a proposed objective and evaluation design; it is not an optimization algorithm implemented in the current code.

What counts as success?

Success has several dimensions. A plan might meet a deadline but contain vague tasks, or look complete while reserving time that is unavailable. I would evaluate the agent with the following weighted score, where each component ranges from 0 to 1:

Score = 0.40D + 0.30F + 0.20A + 0.10T

Component	Weight	Proposed measurement
Deadline coverage (D)	40%	Fraction of required work hours for assignments due within seven days that are scheduled before their deadlines; cap each assignment's coverage at 100% and average across due assignments.
Feasibility (F)	30%	Fraction of proposed study blocks that fit inside available windows, have positive duration, and do not overlap another reservation.
Actionability (A)	20%	Fraction of due assignments with a specific task, due date, and next step tied to the relevant course requirement.
Traceability (T)	10%	Fraction of factual deadline/availability claims linked to a sensor result, and completion claims linked to a successful actuator result.

For test cases with no due assignments, I would exclude D and A and renormalize the remaining weights. Empty or unnecessary schedules should not receive credit for completing work that does not exist.

Acceptance criteria. I would target a score of at least 0.90, subject to hard requirements: no overlapping or unavailable reservations, no invented deadlines, and no claim that a task or file was created unless the corresponding tool succeeded. A high weighted score cannot override these requirements. If there is insufficient time, the agent should explicitly report the unscheduled hours and propose a prioritization decision rather than pretend the plan is complete.

Why these weights? Deadline coverage gets the largest weight because the main purpose is to complete coursework on time. Feasibility comes next because a plan is useful only if I can follow it. Actionability turns broad intentions into concrete work. Traceability receives a smaller numerical weight, while truthful reporting remains mandatory, because I need to understand where the plan came from and trust its action reports.

How I would evaluate it. I would use small, hand-checked scenarios: sufficient free time, insufficient free time, overlapping candidate blocks, missing deadline information, and a simulated file-write failure. I would compare proposed tasks and reservations against the input data and actuator results. I would measure planning quality, not claim that the agent guarantees better grades. These tests and scores have not been run.

The current prototype demonstrates the six tools and a single prioritized planning request. It does not yet implement the full seven-day objective, overlap checks across saved reservations, or this scoring framework. Those are proposed technical goals, as the assignment requests.

Question 2(c): Two additional functionalities

1. Live course and calendar synchronization with adaptive replanning

I would connect the agent to the course learning-management system and my calendar so it can detect changed deadlines, new assignments, and changes in availability. Instead of relying on simulated inputs, it would refresh the relevant information and maintain a persistent plan. If an assignment changes or I miss a study block, it would identify the affected tasks, recalculate remaining work, and propose a revised schedule.

I would want a preview of calendar changes before they are committed, conflict detection against existing reservations, and stable task identifiers so repeated runs do not create duplicate tasks. This is my first priority because maintaining an accurate plan over time is more useful than generating a one-time schedule. I would assess it by checking the time from a deadline change to a proposed revision, deadline coverage, scheduling conflicts, and duplicate records.

2. Evidence-linked literature discovery and study preparation

I would connect the agent to the RSS collection workflow from Question 1 and let it find readings relevant to upcoming assignments. It would filter candidates by their original publication dates, flag articles older than seven days for the weekly discussion, remove duplicate stories, and distinguish original reporting or research from commentary. For selected readings, it would prepare a short evidence-linked brief covering the main claim, supporting evidence, limitations, and connections to the course. It could then propose reading tasks and study blocks through the existing actuators.

This would connect my literature-review habit with my coursework instead of maintaining two separate planning routines. It would support my own reading and writing rather than submit discussion posts for me. I would assess it by publication-date accuracy, relevance to the assignment, duplicate reduction, and whether each summary claim can be verified in the original source. Dates or claims that cannot be verified should be labeled uncertain instead of invented.

References

- LlamaIndex agents
- Google Gemini with LlamaIndex
- RSS Guard: standard feeds and OPML import